

第2章 ARM体系结构

2.1 ARM体系结构简介

2.2 ARM微处理器结构

2.3 ARM微处理器的寄存器结构

2.4 ARM微处理器的异常处理

2.5 ARM的存储器结构

2.6 ARM微处理器的接口

2.1 ARM体系结构简介

ARM (Advanced RISC Machines) 公司1991年成立于英国剑桥，专门从事基于RISC技术芯片设计开发，主要出售芯片设计技术的授权，作为知识产权供应商，不直接芯片生产，转让设计许可由合作公司生产各具特色的芯片，半导体生产商从ARM购买设计的ARM微处理器核，根据不同的应用领域，加入外围电路，形成自己的ARM微处理器芯片进入市场。

目前，全世界有几十家大的半导体公司使用ARM公司的授权，使ARM技术获得更多的第三方工具、制造、软件的支持，又使系统成本降低，使产品更容易进入市场，更具有竞争力。目前，ARM已深入到工业控制、无线通讯、网络应用、消费类电子产品、成像和安全产品各个领域。

ARM的业务模型



架构	处理器家族
ARMv1	ARM1
ARMv2	ARM2 , ARM3
ARMv3	ARM6, ARM7
ARMv4	StrongARM , ARM7TDMI , ARM9TDMI
ARMv5	ARM7EJ , ARM9E , ARM10E , XScale
ARMv6	ARM11 , ARM Cortex-M
ARMv7	ARM Cortex-A , ARM Cortex-M , ARM Cortex-R
ARMv8	

处理器	描述
Cortex-M0	面向低成本，超低功耗的微控制器和深度嵌入应用的非常小的处理器（最小 12K 门电路）
Cortex-M0+	针对小型嵌入式系统的最高能效的处理器，与 Cortex-M0 处理器接近的尺寸大小和编程模式，但是具有扩展功能，如单周期 I/O 接口和向量表重定位功能
Cortex-M1	针对 FPGA 设计优化的小处理器，利用 FPGA 上的存储器块实现了紧耦合内存（ TCM ）。和 Cortex-M0 有相同的指令集
Cortex-M3	针对低功耗微控制器设计的处理器，面积小但是性能强劲，支持可以处理器快速处理复杂任务的丰富指令集。具有硬件除法器 and 乘加指令（ MAC ）。并且， M3 支持全面的调试和跟踪功能，使软件开发者可以快速的开发他们的应用
Cortex-M4	不但具备 Cortex-M3 的所有功能，并且扩展了面向数字信号处理（ DSP ）的指令集，比如单指令多数据指令（ SMID ）和更快的单周期 MAC 操作。此外，它还有一个可选的支持 IEEE754 浮点标准的单精度浮点运算单元
Cortex-M7	针对高端微控制器和数据处理密集的应用开发的高性能处理器。具备 Cortex-M4 支持的所有指令功能，扩展支持双精度浮点运算，并且具备扩展的存储器功能，例如 Cache 和紧耦合存储器（ TCM ）
Cortex-M23	面向超低功耗，低成本应用设计的小尺寸处理器，和 Cortex-M0 相似，但是支持各种增强的指令集和系统层面的功能特性。 M23 还支持 TrustZone 安全扩展
Cortex-M33	主流的处理器的设计，与之前的 Cortex-M3 和 Cortex-M4 处理器类似，但系统设计更灵活，能耗比更高效，性能更高。 M33 还支持 TrustZone 安全扩展

	Application processors	Real-time processors	Microcontroller processors
设计特点	高时钟频率，长流水线，高性能，对媒体处理支持（NEON 指令集扩展），	高时钟频率，较长的流水线，高确定性（中断延迟低）	通常较短的流水线，超低功耗
系统特性	内存管理单元 (MMU), cache memory, ARM TrustZone® 安全扩展	内存保护单元 (MPU), cache memory, 紧耦合内存 (TCM)	内存保护单元(MPU), 嵌套向量中断控制器 (NVIC), 唤醒中断控制器 (WIC), 最新 ARM TrustZone® 安全扩展.
目标市场	移动计算，智能手机，高性能服务器，高端微处理器	工业微控制器，汽车电子，硬盘控制器，基带	微控制器，深度嵌入系统（例如，传感器，MEMS，混合信号 IC, IoT）

采用RISC架构的ARM微处理器特点

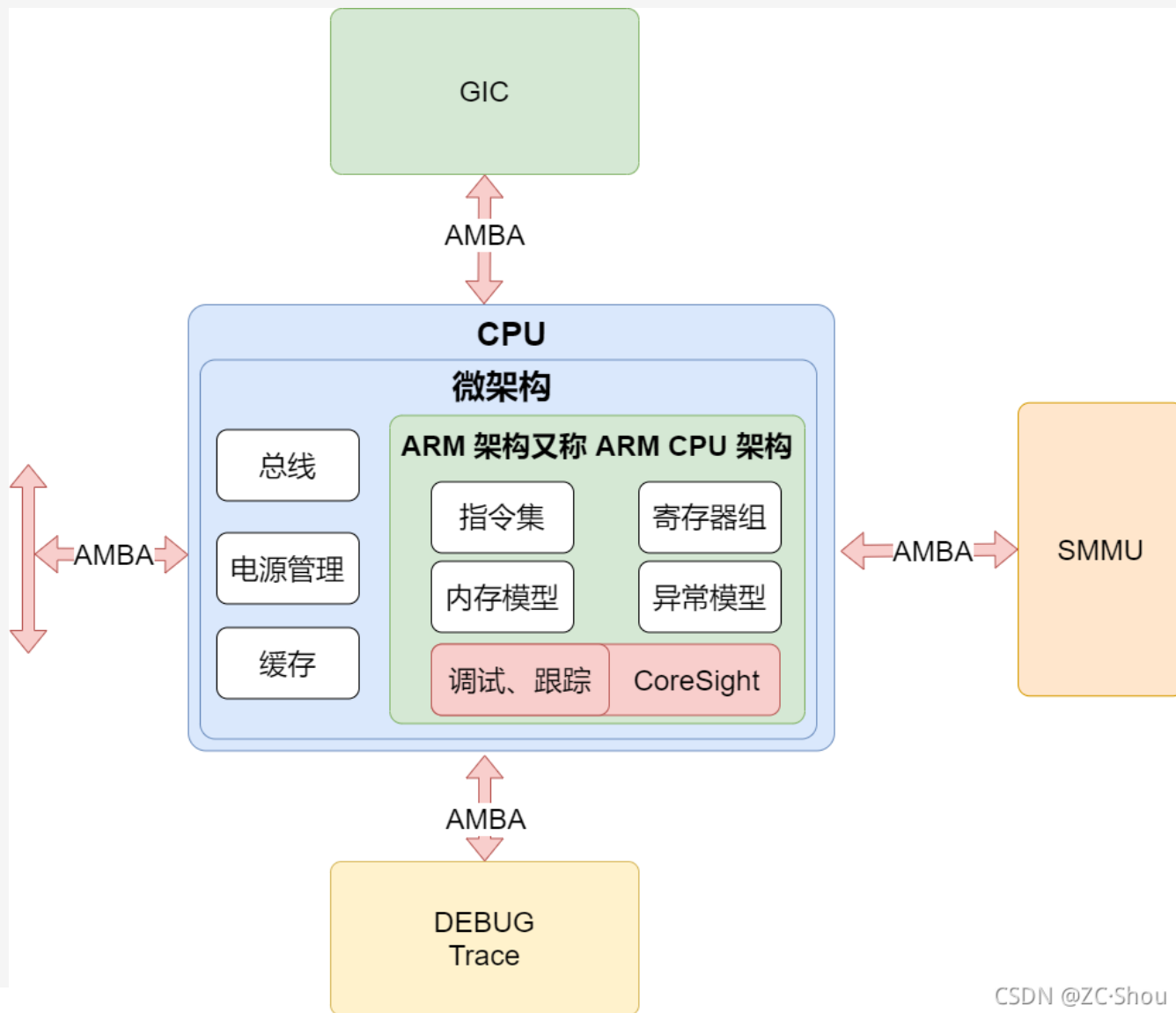
- 支持Thumb(6位)/ARM(32位)双指令集，兼容8位/16位器件。Thumb指令比8位和16位CISC/RISC CPU有更好的代码密度；
- 指令执行采用3级流水线/5级流水线技术；
- 具有指令Cache和数据Cache，大量使用寄存器，指令执行速度更快。大多数数据操作在寄存器中完成。寻址方式灵活简单，执行效率高。指令长度固定(ARM 32位，Thumb 16位)；
- 支持大端格式和小端格式两种方法存储字数据；
- 支持Byte(8位)、Halfword(16位)和Word(32位)数据类型；
- 支持用户、快中断、中断、管理、中止、系统和未定义等7种处理器模式，除用户模式外，其余均为特权模式；

采用RISC架构的ARM微处理器特点

- 处理器芯片上都嵌入了在线仿真ICE-RT逻辑，便于通过JTAG来仿真调试ARM体系结构芯片，可以避免使用昂贵的在线仿真器。另外，在处理器核中还可以嵌入跟踪宏单元ETM，用于监控内部总线，实时跟踪指令和数据的执行；
- 具有片上总线AMBA(Advanced Micro-controller Bus Architecture)。AMBA定义了3组总线：
 1. 先进高性能总线AHB(Advanced High performance Bus)；
 2. 先进系统总线ASB(Advanced System Bus)；
 3. 先进外围总线APB(Advanced Peripheral Bus)。

通过AMBA可以方便地扩充各种处理器及I/O，可以把DSP、其他处理器和I/O(如UART、定时器和接口等)都集成在一块芯片中；

- 采用存储器映像I/O方式，I/O端口作为特殊的存储器地址；

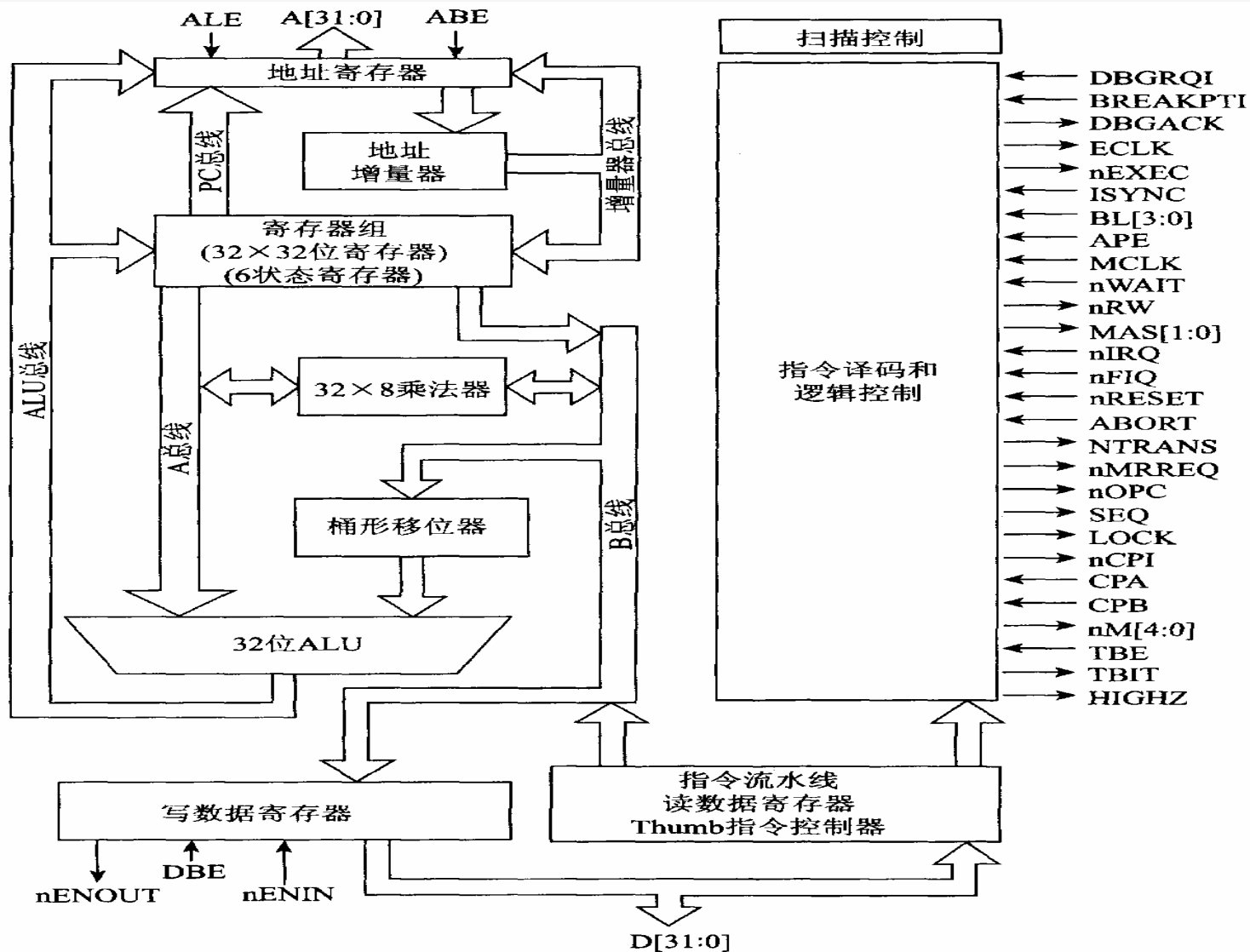


采用RISC架构的ARM微处理器特点

- 具有协处理器接口。ARM允许接16个协处理器，如CP15用于系统控制，CP14用于调试控制器；
- 采用了降低电源电压，可工作在3.0V以下；减少门的翻转次数，当某个功能电路不需要时禁止门翻转；减少门的数目，即降低芯片的集成度；降低时钟频率等一些措施降低功耗；
- 体积小、低成本、高性能

ARM微处理器包括ARM7、ARM9、ARM9E、ARM10E、SecurCore、以及Intel的StrongARM、XScale和其它厂商基于ARM体系结构的处理器，除了具有ARM体系结构的共同特点以外，每一个系列的ARM微处理器都有各自的特点和应用领域。

典型的ARM体系结构方框图



典型ARM体系结构的主要部件

1. ALU

ARM体系结构的ALU与常用的ALU逻辑结构基本相同,由两个操作数锁存器、加法器、逻辑功能、结果及零检测逻辑构成。

2. 桶形移位寄存器

ARM采用了 32×32 位桶形移位寄存器,左移/右移 n 位、环移 n 位和算术右移 n 位等都可一次完成,可有效减少移位的延迟时间。

3. 高速乘法器

ARM为了提高运算速度,采用2位乘法的方法,2位乘法可根据乘数的2位来实现“加 - 移位”运算。ARM的高速乘法器采用 32×8 位的结构,完成 32×2 位乘法也只需5个时钟周期。

典型ARM体系结构的主要部件

4. 浮点部件

在ARM体系结构中，浮点部件作为选件可根据需要选用，FPA10浮点加速器以协处理器方式与ARM相连，并通过协处理器指令的解释来执行。浮点的Load/Store指令使用频率要达到67%，故FPA10内部也采用Load/Store结构，有8个80位浮点寄存器组，指令执行也采用流水线结构。

5. 控制器

ARM的控制器采用硬接线的可编程逻辑阵列PLA，其输入端有14根、输出端有40根，分别控制乘法器、协处理器及地址寄存器、ALU和移位器等。

6. 寄存器

ARM有37个寄存器，31个通用32位寄存器和6个状态寄存器。

2. 2 ARM微处理器结构

2.2.1 ARM7微处理器

2.2.2 ARM9微处理器

2.2.3 ARM9E微处理器

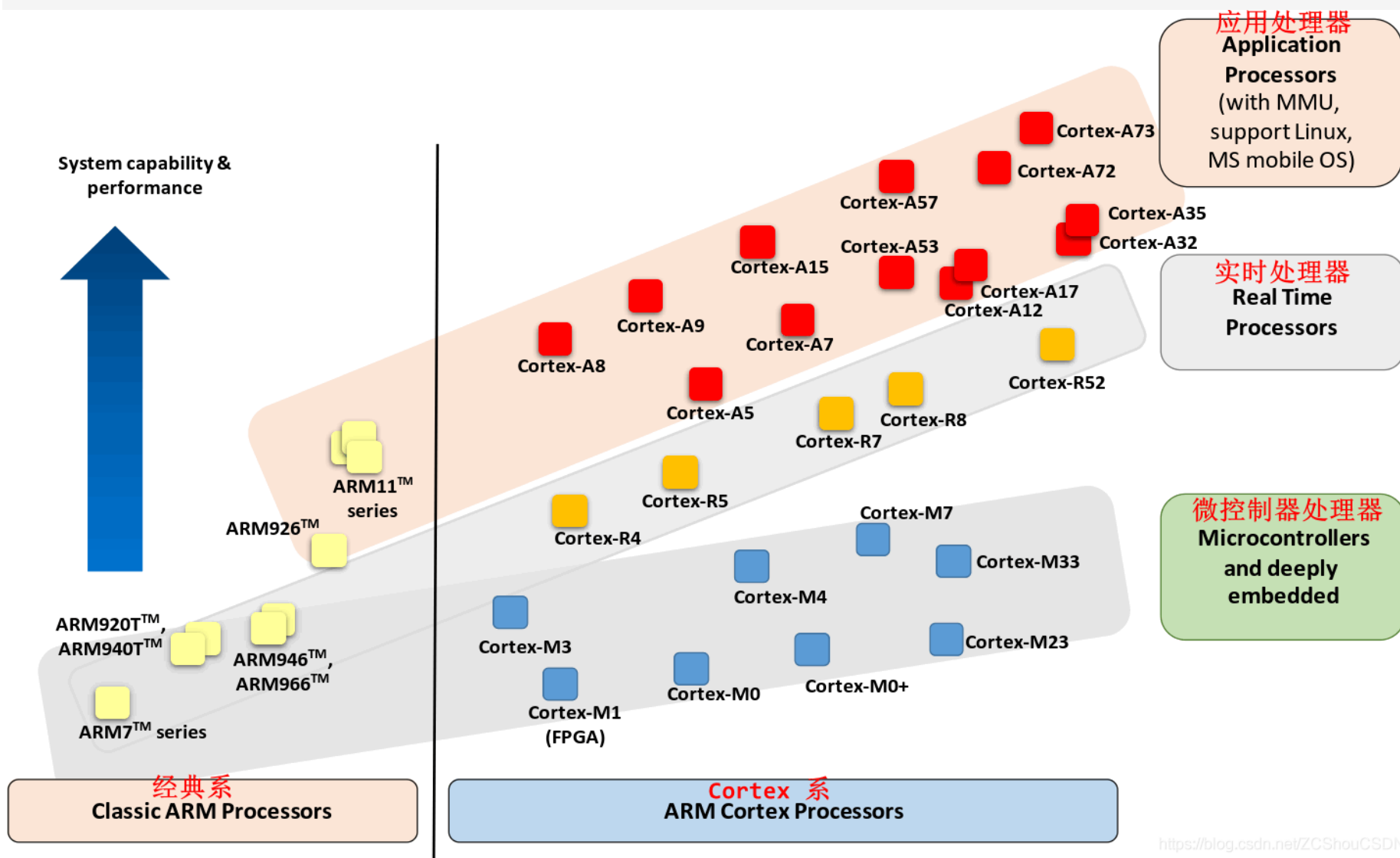
2.2.4 ARM10E微处理器

2.2.5 SecurCore微处理器

2.2.6 StrongARM微处理器

2.2.7 XScale微处理器

总结来说，ARM 处理器产品分为经典ARM处理器系列和最新的Cortex处理器系列。



2.2.1 Classic系列微处理器

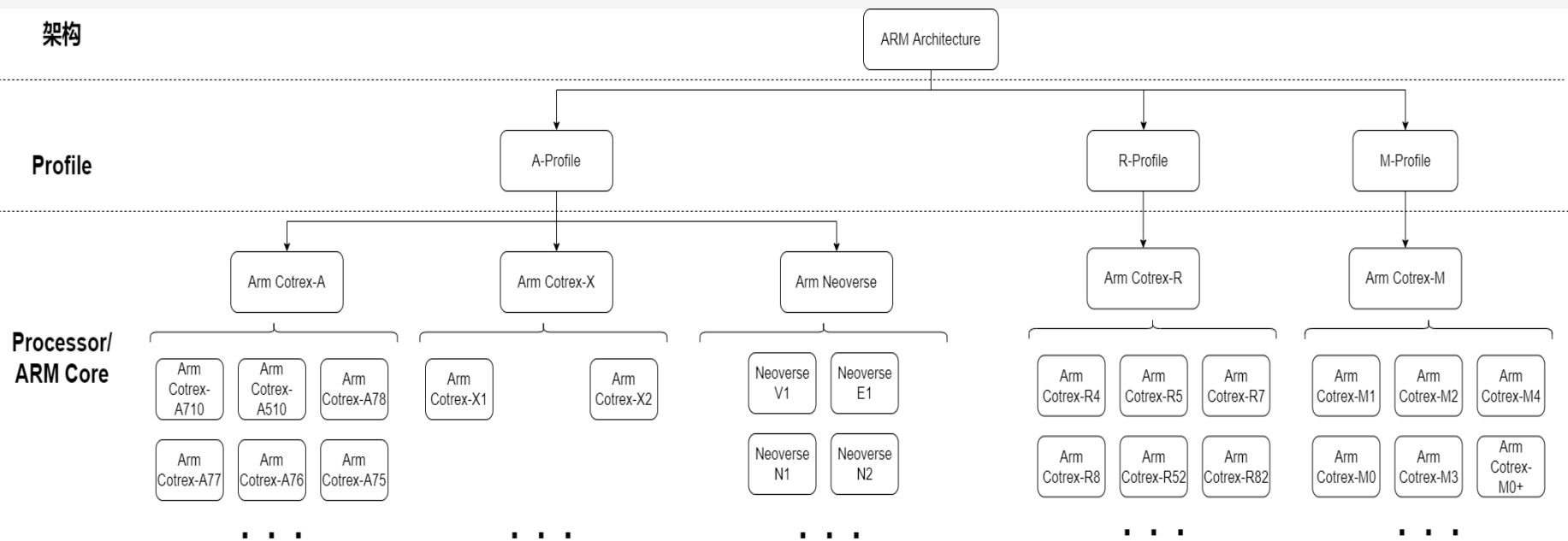
- ARM7微处理器系列：1994年推出，使用范围最广的 32 位嵌入式处理器系列。0.9MIPS/MHz的三级流水线和冯诺依曼结构。ARM7系列包括ARM7TDMI、ARM7TDMI-S、带有高速缓存处理器宏单元的ARM720T。该系列处理器提供Thumb 16位压缩指令集和EmbeddedICE软件调试方式，适用于更大规模的SoC设计中。ARM7TDMI基于ARM体系结构V4版本，是目前低端的ARM核。
- ARM9微处理器系列：ARM9采用哈佛体系结构，指令和数据分属不同的总线，可以并行处理。在流水线上，ARM7是三级流水线，ARM9是五级流水线。由于结构不同，ARM7的执行效率低于ARM9。基于Arm9内核的处理器，是具有低功耗，高效率的开发平台。广泛用于各种嵌入式产品。它主要应用于音频技术以及高档工业级产品，可以跑Linux以及Wince等高级嵌入式系统，可以进行界面设计，做出人性化的人机互动界面，像一些网络产品和手机产品。
- ARM9E微处理器系列：ARM9E中的E就是Enhance instructions，意思是增强型DSP指令，说明了ARM9E其实就是ARM9的一个扩充，变种。ARM9E系列微处理器提供了增强的DSP处理能力，很适合于那些需要同时使用DSP和微控制器的应用场合。
- ARM10E微处理器系列：ARM10E系列微处理器为可综合处理器，使用单一的处理器内核提供了微控制器、DSP、Java应用系统的解决方案，极大的减少了芯片的面积和系统的复杂程度。ARM10E与ARM9E区别在于，ARM10E使用哈佛结构，6级流水线，主频最高可达325MHz，1.35MIPS/MHz。
- ARM11微处理器系列：ARM公司近年推出的新一代RISC处理器，它是ARM新指令架构——ARMv6的第一代设计实现。该系列主要有ARM1136J，ARM1156T2和ARM1176JZ三个内核型号，分别针对不同应用领域。

2.2.1 Cortex系列微处理器

ARM公司在经典处理器ARM11以后的产品改用Cortex命名，并分成A、R和M三类，旨在为各种不同的市场提供服务。Cortex系列属于ARMv7架构，由于应用领域不同，基于v7架构的Cortex处理器系列所采用的技术也不相同，基于v7A的称为Cortex-A系列，基于v7R的称为Cortex-R系列，基于v7M的称为Cortex-M系列。

- **Application Processors (应用处理器)**：面向移动计算，智能手机，服务器等市场的高端处理器。这类处理器运行在很高的时钟频率（超过1GHz），支持像Linux，Android，MS Windows和移动操作系统等完整操作系统需要的内存管理单元（MMU）。如果规划开发的产品需要运行上述其中的一个操作系统，你需要选择ARM 应用处理器。
- **Real-time Processors (实时处理器)**：面向实时应用的高性能处理器系列，例如硬盘控制器，汽车传动系统和无线通讯的基带控制。多数实时处理器不支持MMU，不过通常具有MPU、Cache和其他针对工业应用设计的存储器功能。实时处理器运行在比较高的时钟频率（例如200MHz 到 >1GHz ），响应延迟非常低。虽然实时处理器不能运行完整版本的Linux和Windows操作系统，但是支持大量的实时操作系统（RTOS）。
- **Microcontroller Processors (微控制器处理器)**：微控制器处理器通常设计成面积很小和能效比很高。通常这些处理器的流水线很短，最高时钟频率很低（虽然市场上有此类的处理器可以运行在200Mhz之上）。并且，新的Cortex-M处理器家族设计的非常容易使用。因此，ARM 微控制器处理器在单片机和深度嵌入式系统市场非常成功和受欢迎。
- Cortex-M 处理器家族更多的集中在低性能端，但是这些处理器相比于许多微控制器使用的传统处理器性能仍然很强大。例如，Cortex-M4 和 Cortex-M7 处理器应用在许多高性能的微控制器产品中，最大的时钟频率可以达到400Mhz。

架构

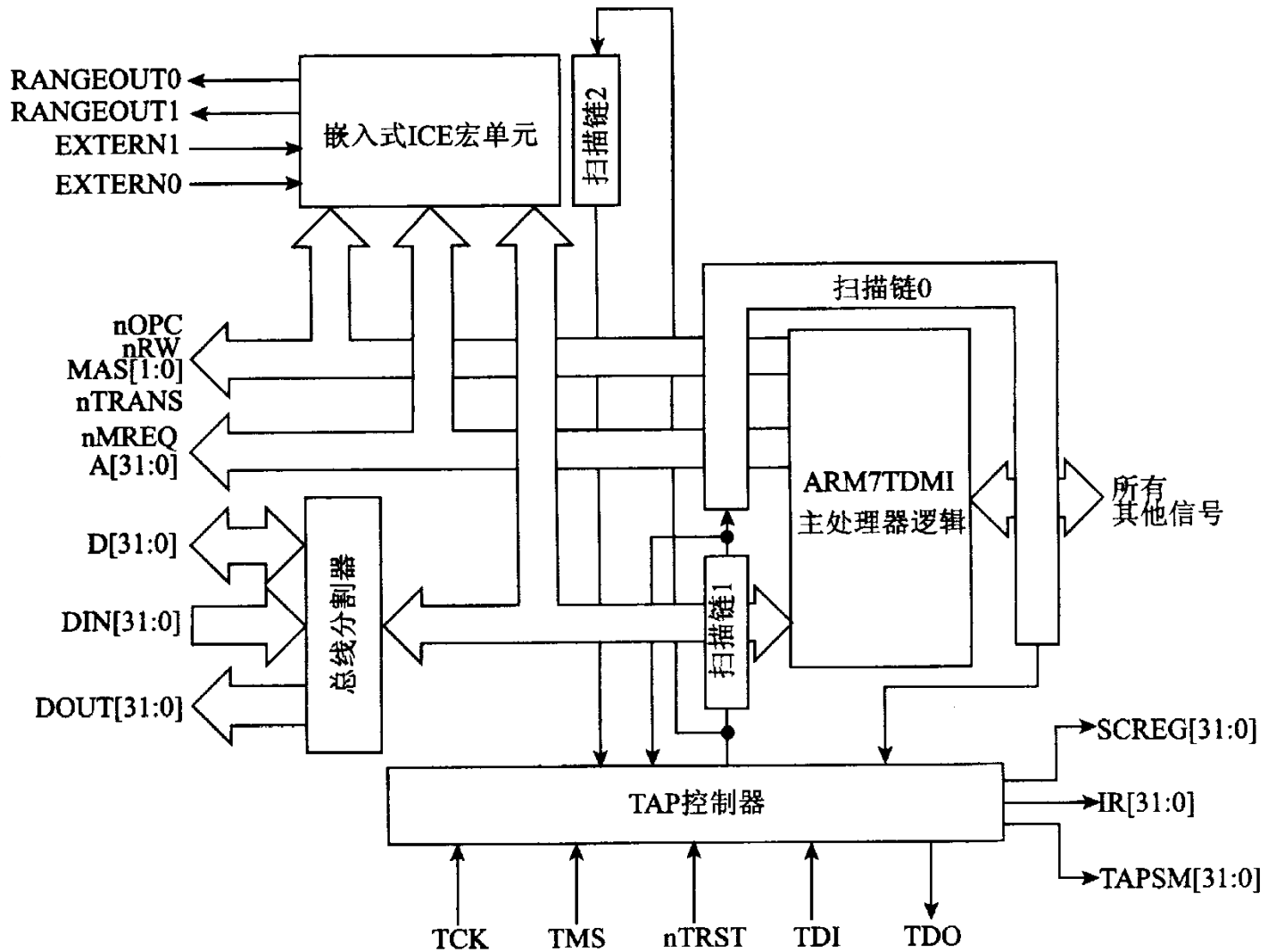


2.2.1 ARM7微处理器

ARM7系列微处理器包括ARM7TDMI、ARM7TDMI-S、ARM720T、ARM7EJ几种类型。

其中，ARM7TMDI是目前使用最广泛的32位嵌入式RISC处理器，主频最高可达130 MIPS；采用能够提供0.9MIPS/MHz的三级流水线结构；内嵌硬件乘法器（Multiplier）；支持16位压缩指令集Thumb；嵌入式ICE；支持片上Debug；支持片上断点和调试点；指令系统与ARM9系列、ARM9E系列和ARM10E系列兼容；支持Windows CE、Linux、Palm OS等操作系统。典型产品如Samsung公司的S3C4510B。

1. ARM7TDMI 处理器内核



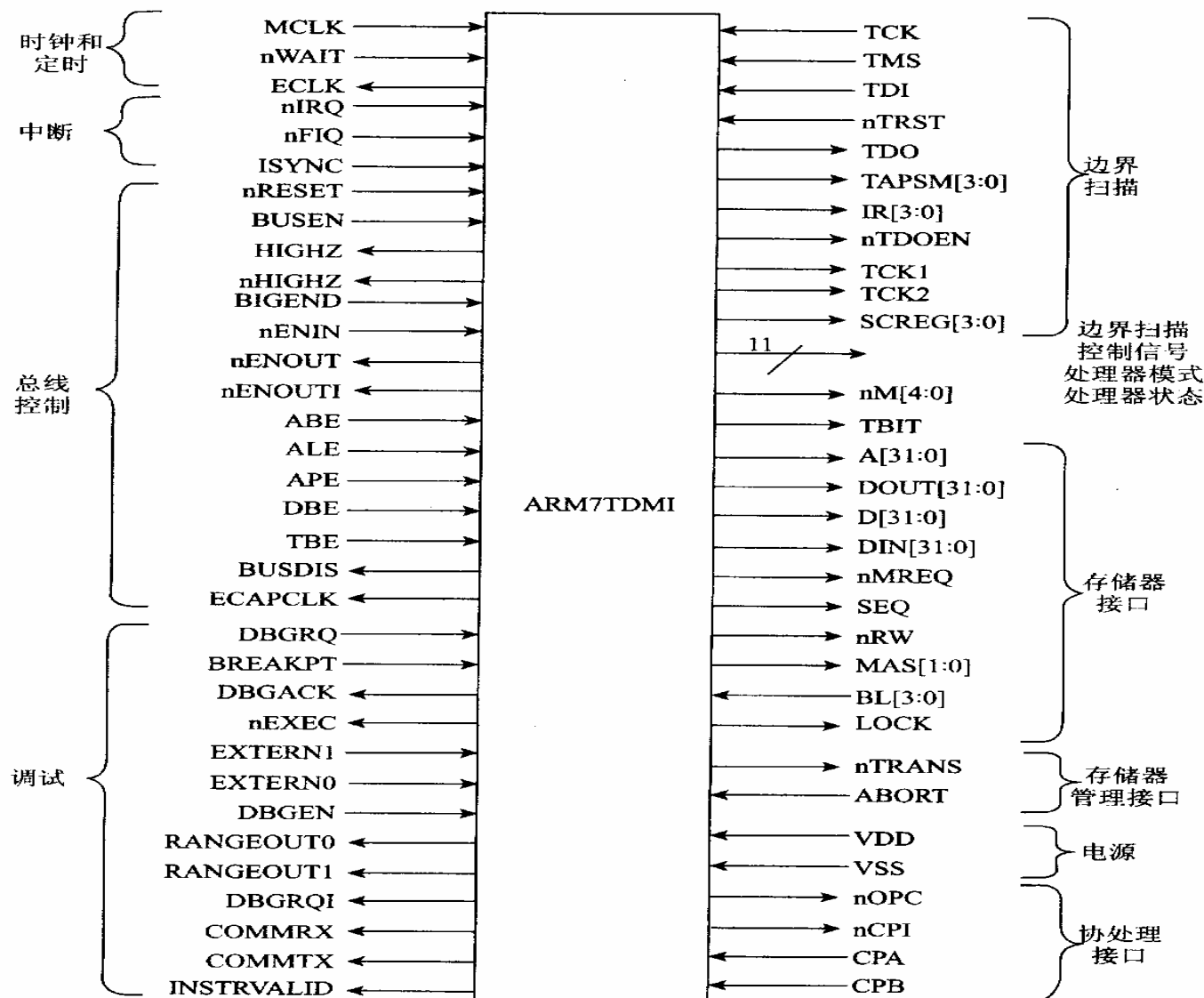
ARM7TDMI 处理器组成主要部件

ARM7TDMI提供了存储器接口、MMU接口、协处理器接口和调试接口，以及时钟与总线等控制信号，如图2.2.2所示。

存储器接口包括了32位地址A[31:0]、双向32位数据总线D[31:0]、单向32位数据总线DIN[31:0]与DOUT[31:0]、以及存储器访问请求MREQ、地址顺序SEQ、存储器访问控制MAS[1:0]和数据锁存控制BL[3:0]等控制信号。

ARM7TDMI处理器内核也可ARM7TDMI-S软核（Softcore）形式向用户提供。同时，提供多种组合选择，例如可以省去嵌入式ICE宏单元等。

ARM7TDMI 处理器外部引脚组成

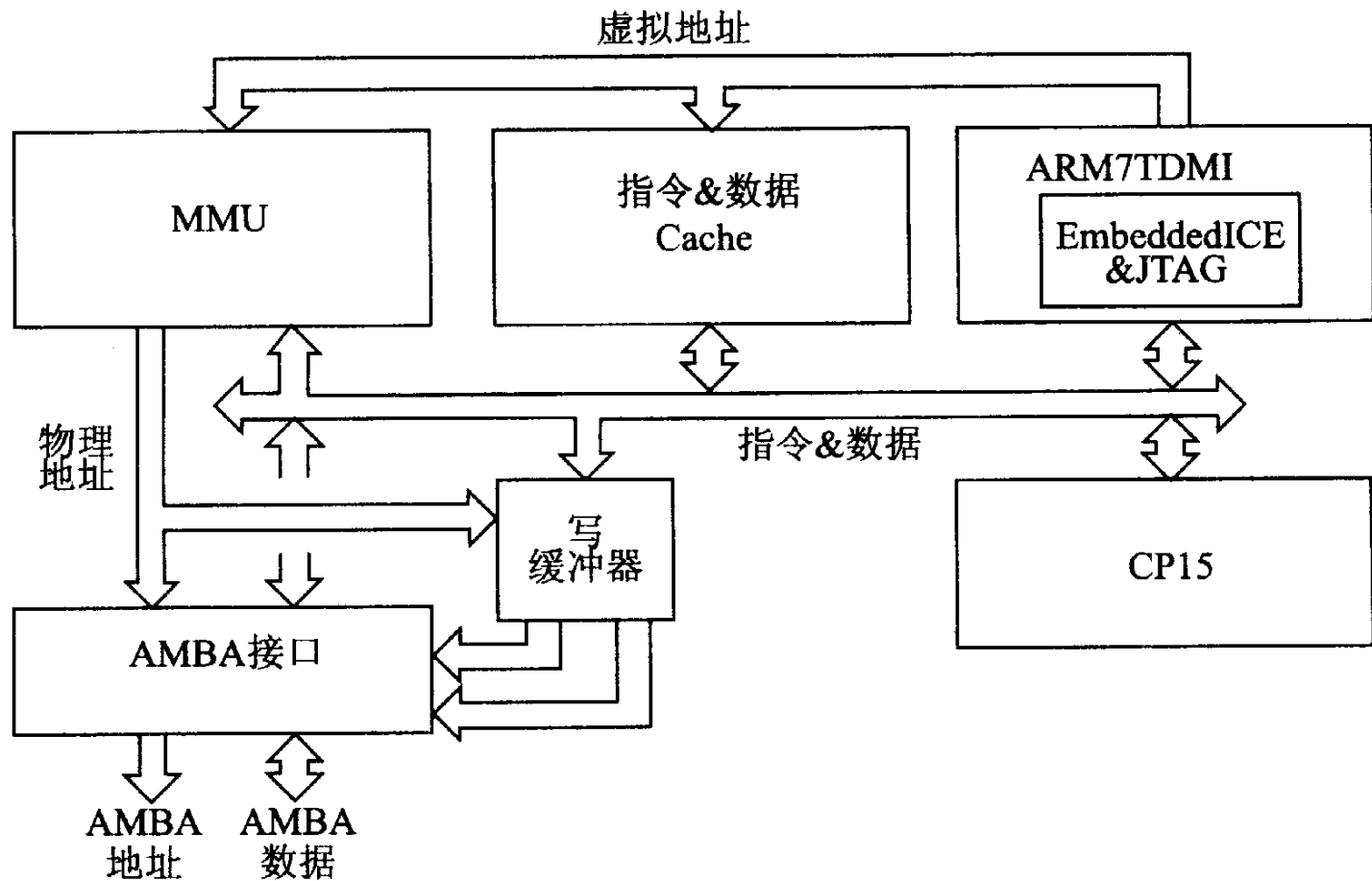


2. ARM720T/ARM740T处理器内核

ARM720T处理器内核是在ARM7TDMI处理器内核基础上，增加8KB的数据与指令Cache，支持段式和页式存储的MMU(Memory Management Unit)、写缓冲器及AMBA(Advanced Microcontroller Bus Architecture)接口而构成,如下图所示.

ARM740T处理器内核与ARM720T处理器内核相比，结构基本相同，ARM740T处理器核没有存储器管理单元MMU，不支持虚拟存储器寻址，而是用存储器保护单元来提供基本保护和Cache的控制。合适低价格低功耗的嵌入式应用。

ARM720T内核结构

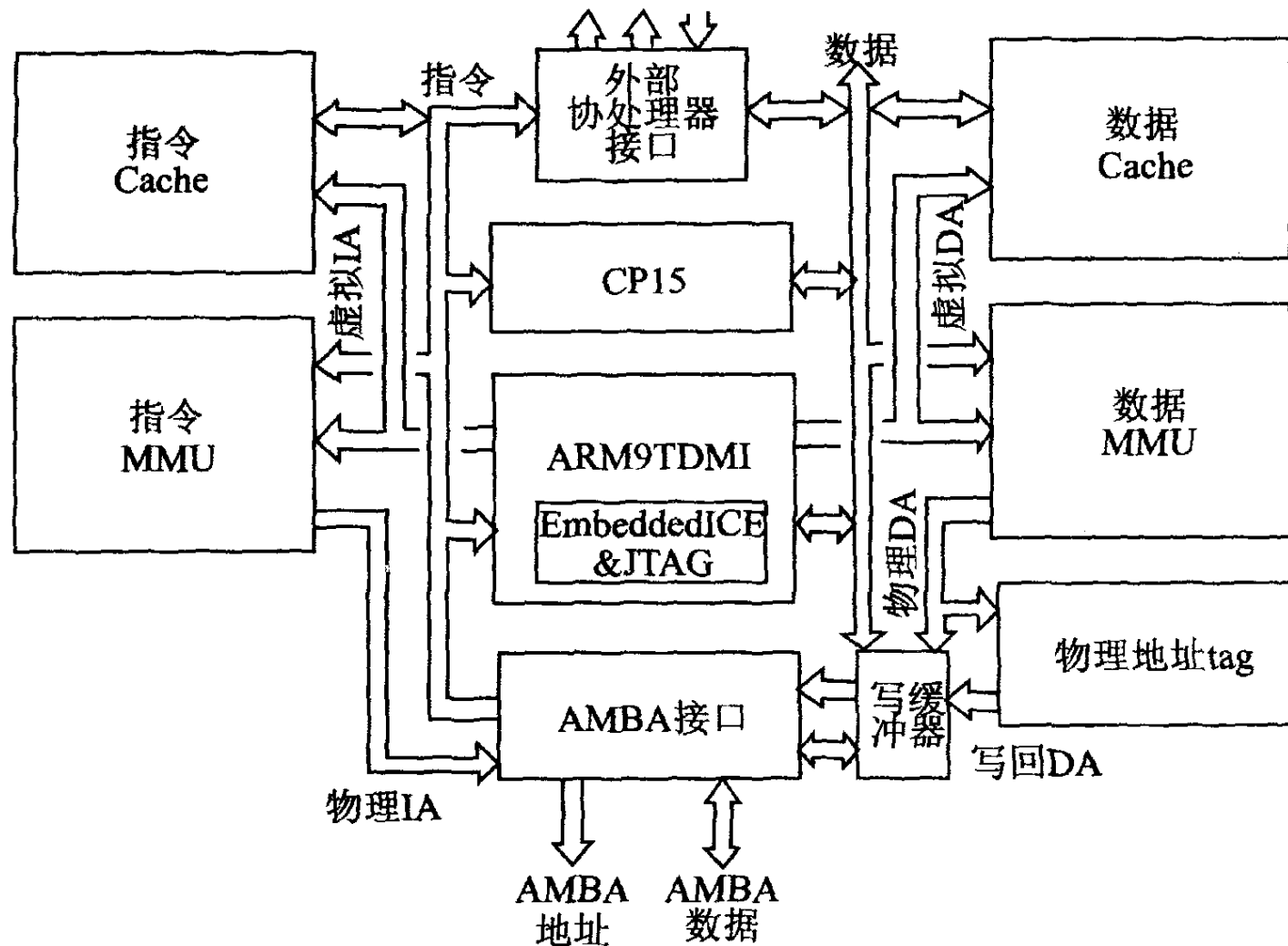


2.2.2 ARM9微处理器

ARM9系列微处理器包含ARM920T、ARM922T和ARM940T几种类型，可以在高性能和低功耗特性方面提供最佳的性能。采用5级整数流水线，指令执行效率更高。提供1.1MIPS/MHz的哈佛结构。支持数据Cache和指令Cache，具有更高的指令和数据处理能力。支持32位ARM指令集和16位Thumb指令集。支持32位的高速AMBA总线接口。全性能的MMU，支持Windows CE、Linux、Palm OS等多种主流嵌入式操作系统。MPU支持实时操作系统。

ARM920T处理器核在ARM9TDMI处理器内核基础上，增加了分离式的指令Cache和数据Cache，并带有相应的存储器管理单元I-MMU和D-MMU、写缓冲器及AMBA接口等，如图2.2.4所示。

ARM920T内核结构



ARM940T处理器核

ARM940T处理器核采用了ARM9TDMI处理器内核，是ARM920T处理器核的简化版本，没有存储器管理单元MMU，不支持虚拟存储器寻址，而是用存储器保护单元来提供存储保护和Cache控制。

ARM9系列微处理器主要应用于无线通信设备、仪器仪表、安全系统、机顶盒、高端打印机、数字照相机和数字摄像机等。典型产品如Samsung公司的S3C2410A。

2. 2. 3 ARM9E微处理器

ARM9E系列微处理器包含ARM926EJ-S、ARM946E-S和ARM966E-S几种类型，使用单一的处理器内核提供了微控制器、DSP、Java应用系统的解决方案。ARM9E系列微处理器提供了增强的DSP处理能力，很适合于那些需要同时使用DSP和微控制器的应用场合。

ARM9E系列微处理器支持DSP指令集，适合于需要高速数字信号处理的场合。ARM9E系列微处理器采用5级整数流水线，支持32位ARM指令集和16位Thumb指令集，支持32位的高速AMBA总线接口，支持VFP9浮点处理协处理器，MMU支持Windows CE、Linux、Palm OS等多种主流嵌入式操作系统，MPU支持实时操作系统，支持数据Cache和指令Cache，主频最高可达300MIPS。

ARM9系列微处理器主要应用于无线终端设备、数字消费品、成像设备、工业控制、存储设备和网络设备等领域。

2.2.4 ARM10E微处理器

ARM10E系列微处理器包含ARM1020E、ARM1022E和ARM1026EJ-S几种类型，由于采用了新的体系结构，与同等的ARM9器件相比较，在同样的时钟频率下，性能提高了近50%。同时采用了两种先进的节能方式，使其功耗极低。

ARM10E系列微处理器支持DSP指令集，适合于需要高速数字信号处理的场合。采用6级整数流水线，支持32位ARM指令集和16位Thumb指令集，支持32位的高速AMBA总线接口，支持VFP10浮点处理协处理器，MMU支持Windows CE、Linux、Palm OS等多种主流嵌入式操作系统，支持数据Cache和指令Cache，内嵌并行读/写操作部件，主频最高可达400MIPS。

ARM10E系列微处理器主要应用于下一代无线设备、数字消费品、成像设备、工业控制、通信和信息系统等领域。

2.2.5 SecurCore微处理器

SecurCore系列微处理器包含SecurCore SC100、SecurCore SC110、SecurCore SC200和SecurCore SC210几种类型，提供了完善的32位RISC技术的**安全解决方案**。

SecurCore系列微处理器除了具有ARM体系结构各种主要特点外，在**系统安全方面**：带有灵活的保护单元，以确保操作系统和应用数据的安全；采用**软内核技术**，防止外部对其进行扫描探测；可集成用户自己的安全特性和其他协处理器。

SecurCore系列微处理器主要应用于如电子商务、电子政务、电子银行业务、网络和认证系统等一些对安全性要求较高的应用产品及应用系统。

2.2.6 StrongARM微处理器

Intel StrongARM处理器是采用ARM体系结构高度集成的32位RISC微处理器，采用在软件上兼容ARMv4体系结构、同时采用具有Intel技术优点的体系结构。典型产品如SA110 处理器、SA1100、SA1110PDA系统芯片和SA1500多媒体处理器芯片等。例如其中的Intel StrongARM SA-1110 微处理器是一款集成了32位StrongARM RISC处理器核、系统支持逻辑、多通信通道、LCD控制器、存储器和PCMCIA控制器以及通用I/O口的高集成度通信控制器。该处理器最高可在206 MHz下运行。SA-1110有一个大的指令Cache和数据Cache、内存管理单元（MMU）和读/写缓存。存储器总线可以和包括SDRAM、SMROM 和类似SRAM的许多器件相接。软件与ARM V4结构处理器家族相兼容。

Intel StrongARM处理器是便携式通讯产品和消费类电子产品的理想选择。

2.2.7 XScale微处理器

Intel XScale微体系结构提供了一种全新的、高性价比、低功耗且基于ARMv5TE体系结构的解决方案，支持16位Thumb指令和DSP扩充。基于XScale技术开发的微处理器，可用于手机、便携式终端(PDA)、网络存储设备、骨干网(BackBone)路由器等。

XScale处理器的处理速度是Intel StrongARM处理速度的两倍，数据Cache的容量从8KB增加到32KB，指令Cache的容量从16KB增加到32KB，微小数据Cache的容量从512B增加到2KB；为了提高指令的执行速度，超级流水线结构由5级增至7级；新增乘/加法器MAC和特定的DSP型协处理器，以提高对多媒体技术的支持；动态电源管理，使XScale处理器的时钟可达1GHz、功耗1.6W，并能达到1200MIPS。

XScale微处理器

XScale微处理器架构经过专门设计，核心采用了英特尔先进的0.18 μ m工艺技术制造；具备低功耗特性，适用范围从0.1mW~1.6W。同时，它的时钟工作频率将接近1GHz。XScale与StrongARM相比，可大幅降低工作电压并且获得更高的性能。具体来讲，在目前的StrongARM中，在1.55V下可以获得133MHz的工作频率，在2.0V下可以获得206MHz的工作频率；而采用XScale后，在0.75V时工作频率达到150MHz，在1.0V时工作频率可以达到400MHz，在1.65V下工作频率则可高达800MHz。超低功率与高性能的组合使Intel XScale适用于广泛的互联网接入设备，在因特网的各个环节中，从手持互联网设备到互联网基础设施产品，Intel XScale都表现出了令人满意的处理性能。

Intel采用XScale架构的嵌入式处理器典型产品有PXA25x、PXA26x和PXA27x系列。

2.3 ARM微处理器的寄存器结构

2.3.1 处理器的运行模式

2.3.2 处理器的工作状态

2.3.3 处理器的寄存器组织

2.3.4 Thumb状态的寄存器集

ARM微处理器的寄存器结构

- ARM处理器共有37个寄存器，被分为若干个组（BANK），这些寄存器包括：
- 31个通用寄存器，包括程序计数器（PC指针），均为32位的寄存器。
- 6个状态寄存器，用以标识CPU的工作状态及程序的运行状态，均为32位，目前只使用了其中的一部分。

2.3.1 处理器的运行模式

ARM微处理器支持7种运行模式，分别为：

- **usr(用户模式)**：ARM处理器正常程序执行模式。
- **fiq(快速中断模式)**：用于高速数据传输或通道处理
- **irq(外部中断模式)**：用于通用的中断处理
- **svc(管理模式)**：操作系统使用的保护模式
- **abt (数据访问终止模式)**：当数据或指令预取终止时进入该模式，可用于虚拟存储及存储保护。
- **sys(系统模式)**：运行具有特权的操作系统任务。
- **und(未定义指令中止模式)**：当未定义的指令执行时进入该模式，可用于支持硬件协处理器的软件仿真。

ARM微处理器支持7种运行模式

CPSR [4:0]	模式	用途	可访问的寄存器
10000	用户	正常用户模式, 程序正常执行模式	PC, R14-R0, CPSR
10001	FIQ	处理快速中断,支持高速数据传 送或通道处理	PC, R14_fiq-R8_fiq, R7-R0, CPSR, SPSR_fiq
10010	IRQ	处理普通中断	PC, R14_irq-R13_fiq, R12- R0, CPSR, SPSR_irq
10011	SVC	操作系统保护模式 处理软件中断 (SWI)	PC, R14_svc-R13_svc, R12- R0, CPSR, SPSR_svc
10111	中止	处理存储器故障、实现虚拟存储 器和存储器保护	PC, R14_abt-R13_abt, R12- R0, CPSR, SPSR_abt
11011	未定义	处理未定义的指令陷阱, 支持硬 件协处理器的软件仿真	PC, R14_und-R13_und, R12-R0, CPSR, SPSR_und
11111	系统	运行特权操作系统任务	PC, R14-R0, CPSR

处理器的运行模式

ARM微处理器的运行模式可以通过软件改变，也可以通过外部中断或异常处理改变。

大多数的应用程序运行在用户模式下，当处理器运行在用户模式下时，某些被保护的系统资源是不能被访问的。

除用户模式外，其余所有6种模式称为非用户模式,或特权模式;其中除去用户模式和系统模式外的5种又称为异常模式,常用于处理中断或异常,以及需要访问受保护的系统资源等情况。

ARM处理器在每种模式下均有一组相应的寄存器与之对应。在任意一种处理器模式下，可访问的寄存器包括15个通用寄存器(R0 ~ R14)、一至二个状态寄存器和程序计数器。在所有寄存器中，有些是在7种处理器模式下共用的同一个物理寄存器，而有些寄存器则是在不同的处理器模式下有不同的物理寄存器。

2.3.2 处理器的工作状态

ARM处理器有**32位ARM**和**16位Thumb**两种工作状态。ARM状态下执行字ARM指令，在Thumb状态下执行半字Thumb指令。ARM处理器可切换两种工作状态,不影响处理器的模式或寄存器内容。

(1) 当操作数寄存器的状态位（位[0]）**为1时**,执行BX指令**进入Thumb状态**。如处理器在Thumb状态进入异常,则当异常处理(IRQ、FIQ、Undef、Abort和SWI)返回时,自动转到Thumb状态。

(2) 当操作数寄存器的状态位（位 [0] ）**为0时**，执行BX指令**进入ARM状态**，处理器进行异常处理（IRQ、FIQ、Reset、Undef、Abort和SWI）。在此情况下，把PC放入异常模式链接寄存器中。从异常向量地址开始执行也可以进入ARM状态。

2.3.3 处理器的寄存器组织

ARM处理器的37个寄存器被安排成部分重叠的组，不能在任何模式都可以使用，寄存器的使用与处理器状态和工作模式有关。如图2.3.1所示，每种处理器模式使用不同的寄存器组。其中15个通用寄存器（R0 ~ R14）、1或2个状态寄存器和程序计数器是通用的。

处理器的寄存器组织

模式						
特权模式 异常模式						
用户	系统	管理	中止	未定义	中断	快中断
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

1. 通用寄存器

通用寄存器 (R0 ~ R15) 可分成不分组寄存器R0 ~ R7、分组寄存器R8 ~ R14和程序计数器R15三类。

(1) 不分组寄存器R0 ~ R7

不分组寄存器R0 ~ R7是真正的通用寄存器，可以工作在所有的处理器模式下，没有隐含的特殊用途。

(2) 分组寄存器R8 ~ R14

分组寄存器R8 ~ R14取决于当前的处理器模式，每种模式有专用的分组寄存器用于快速异常处理。

寄存器R8 ~ R12可分为两组物理寄存器。一组用于FIQ模式，另一组用于除FIQ以外的其他模式。第1组访问R8_fiq ~ R12_fiq，允许快速中断处理。第二组访问R8_usr ~ R12_usr，寄存器R8 ~ R12没有任何指定的特殊用途。

通用寄存器

寄存器R13 ~ R14可分为6个分组的物理寄存器。1个用于用户模式和系统模式，而其他5个分别用于svc、abt、und、irq和fiq五种异常模式。访问时需要指定它们的模式，如：R13_<mode>，R14_<mode>；其中：<mode>可以从usr、svc、abt、und、irq和fiq六种模式中选取一个。

寄存器R13通常用作堆栈指针，称作SP。每种异常模式都有自己的分组R13。通常R13应当被初始化成指向异常模式分配的堆栈。在入口处，异常处理程序将用到的其他寄存器的值保存到堆栈中；返回时，重新将这些值加载到寄存器。这种异常处理方法保证了异常出现后不会导致执行程序的状态不可靠。

寄存器R14用作子程序链接寄存器，也称为链接寄存器LK。当执行带链接分支（BL）指令时，得到R15的备份。

通用寄存器

在其他情况下，将R14当做通用寄存器。类似地，当中断或异常出现时，或当中断或异常程序执行BL指令时，相应的分组寄存器R14_svc、R14_irq、R14_fiq、R14_abt和R14_und用来保存R15的返回值。

FIQ模式有7个分组的寄存器R8 ~ R14，映射为R8_fiq ~ R14_fiq。在ARM状态下，许多FIQ处理没必要保存任何寄存器。User、IRQ、Supervisor、Abort和Undefined模式每一种都包含两个分组的寄存器R13和R14的映射，允许每种模式都有自己的堆栈和链接寄存器。

(3) 程序计数器R15

寄存器R15为程序计数器(PC)。ARM状态,位[1:0]为0,位[31:2]保存PC。

在Thumb状态,位[0]为0,位[31:1]保存PC。R15虽然也可用作通用寄存器,但一般不这么使用,因为对R15的使用有一些特殊的限制,当违反了这些限制时,程序的执行结果是未知的。

① 读程序计数器。指令读出的R15的值是指令地址加上8字节。由于ARM指令始终是字对齐的,所以读出结果值的位[1:0]总是0(在Thumb状态下,情况有所变化)。读PC主要用于快速地对临近的指令和数据进行位置无关寻址,包括程序中的位置无关转移。

② 写程序计数器。写R15的通常结果是将写到R15中的值作为指令地址,并以此地址发生转移。由于ARM指令要求字对齐,通常希望写到R15中值的位[1:0]=0b00。

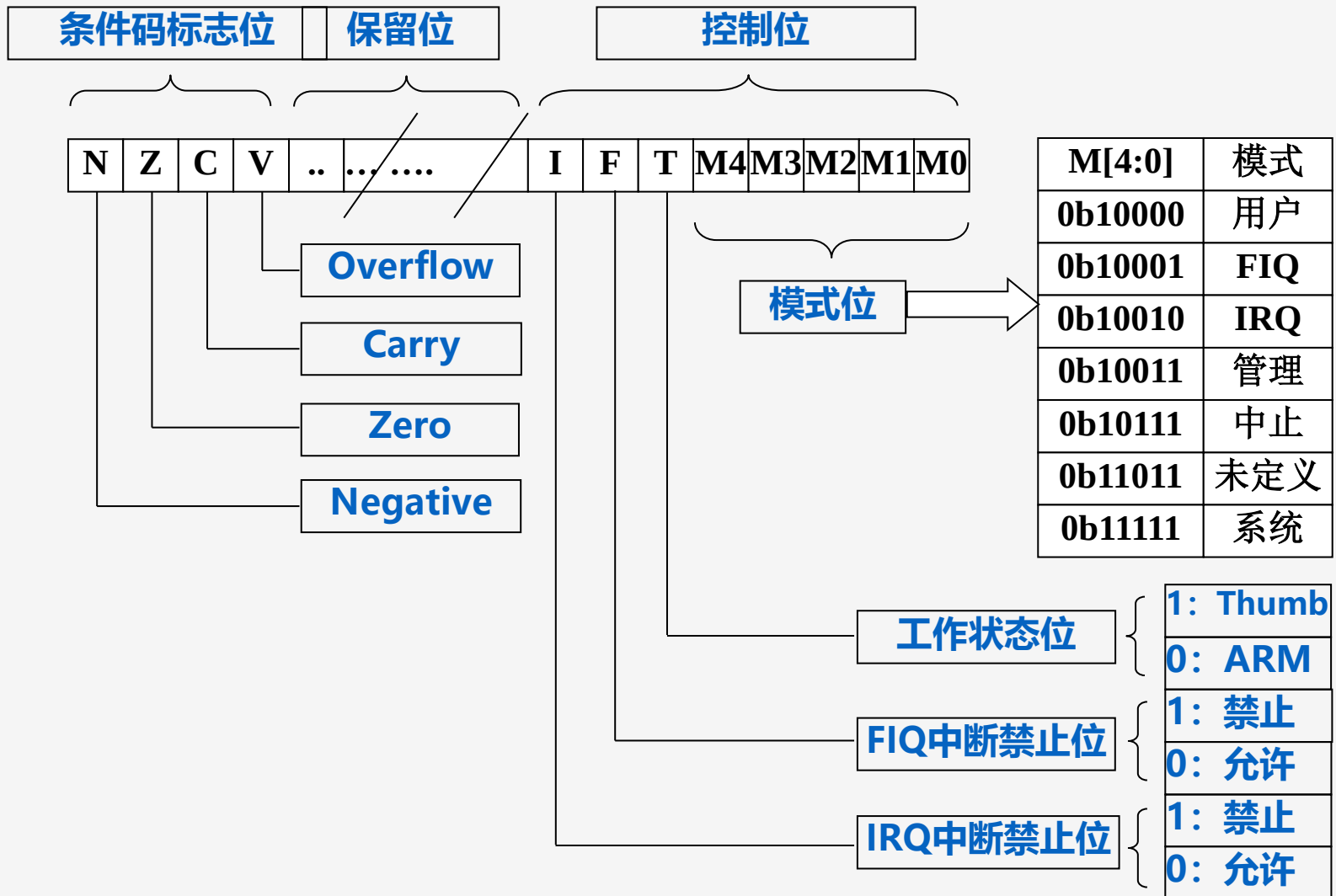
由于ARM体系结构采用多级流水线技术,对于ARM指令集而言,PC总是指向当前指令的下两条指令的地址,即PC的值为当前指令的地址值加8个字节。

2. 程序状态寄存器

寄存器R16用作程序状态寄存器CPSR (Current Program Status Register, 当前程序状态寄存器)。 在所有处理器模式下都可以访问CPSR。CPSR包含**条件码标志、中断禁止位、当前处理器模式及其他状态和控制信息**。每种异常模式都有一个程序状态保存寄存器SPSR (Saved Program Status Register) 。当异常出现SPSR用于保留CPSR的状态。

CPSR和SPSR的格式如下：

程序状态寄存器



(1) 条件码标志

N、Z、C、V (Negative、Zero、Carry、oVerflow) 为条件码标志位，内容可被算术或逻辑运算的结果改变，并决定某条指令是否被执行。ARM状态下，绝大多数的指令都是有条件执行的。Thumb状态下，仅有分支指令是有条件执行的。

通常条件码标志通过执行比较指令 (CMN、CMP、TEQ、TST)、一些算术运算、逻辑运算和传送指令进行修改。

条件码标志的通常含义如下：

- **N**：如果结果是带符号二进制补码，那么，若结果为负数，则N=1；若结果为正数或0，则N=0。
- **Z**：若指令的结果为0，则置1（通常表示比较的结果为“相等”），否则置0。

条件码标志

- **C**: 可用如下4种方法之一设置:

加法 (包括比较指令CMN)。若加法产生进位 (即无符号溢出), 则C置1; 否则置0。

减法 (包括比较指令CMP)。若减法产生借位 (即无符号溢出), 则C置0; 否则置1。

对于结合移位操作的非加法 / 减法指令, C置为移出值的最后1位。

对于其他非加法 / 减法指令, C通常不改变。

- **V**: 可用如下两种方法设置, 即

对于加法或减法指令, 当发生带符号溢出时, V置1, 认为操作数和结果是补码形式的带符号整数。

对于非加法 / 减法指令, V通常不改变。

(2) 控制位

程序状态寄存器PSR (Program Status Register) 的最低8位I、F、T和M[4: 0]用作控制位。当异常出现时改变控制位。处理器在特权模式下时也可由软件改变。

a.中断禁止位

I: 置1, 则禁止IRQ中断; F: 置1, 则禁止FIQ中断。

b.T位

T=0 指示ARM执行; T=1 指示Thumb执行。

c.模式控制位

M4、M3、M2、M1和M0 (M[4:0]) 是模式位, 决定处理器的工作模式, 如表2.3.1所列。

2.3.4 Thumb状态的寄存器集

Thumb状态下的寄存器集如下图所示，是ARM状态下的寄存器集的子集。程序员可以直接访问8个通用寄存器（R0 ~ R7）、PC、SP、LR和CPSR。每一种特权模式都有一组SP、LR和SPSR。

- Thumb状态R0 ~ R7与ARM状态R0 ~ R7是一致的。
- Thumb状态CPSR和SPSR与ARM的状态CPSR和SPSR是一致的。
- Thumb状态SP映射到ARM状态R13。
- Thumb状态LR映射到ARM状态R14。
- Thumb状态PC映射到ARM状态PC（R15）。

Thumb状态下寄存器组织

Thumb状态的通用寄存器和程序计数器

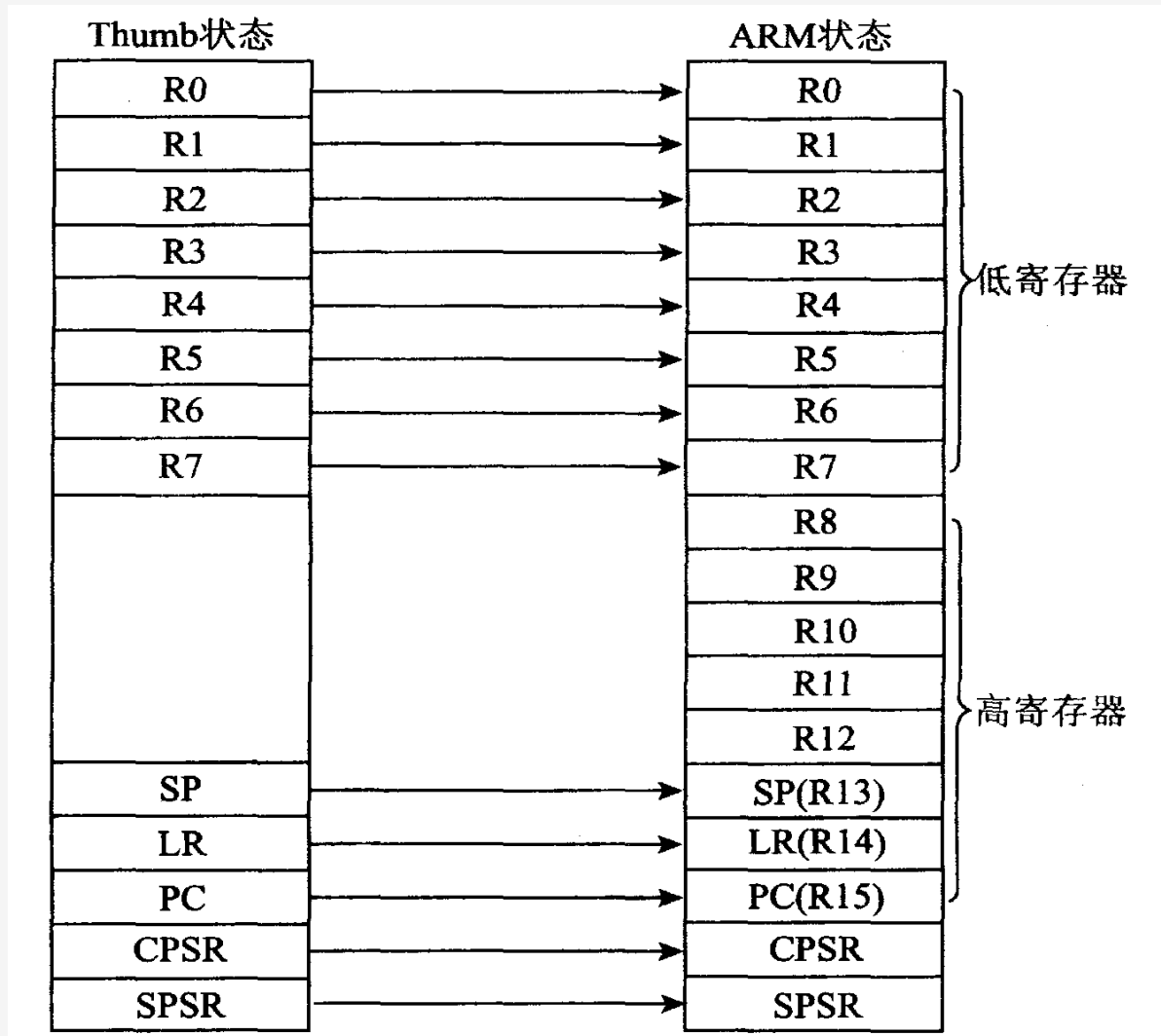
系统和用户	FIQ	管理	中止	IRQ	未定义
R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7
SP	SP_fiq*	SP_svc*	SP_abt*	SP_irq*	SP_und*
LR	LR_fiq*	LR_svc*	LR_abt*	LR_irq*	LR_und*
PC	PC	PC	PC	PC	PC

Thumb状态的程序状态计数器

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	SPSR_fiq	SPSR_svc*	SPSR_abt*	SPSR_irq*	SPSR_und*

* 分组的寄存器。

Thumb状态与ARM状态的寄存器关系



2.4 ARM微处理器的异常处理

2.4.1 ARM体系结构的异常类型

2.4.2 异常类型的含义

2.4.3 异常的响应过程

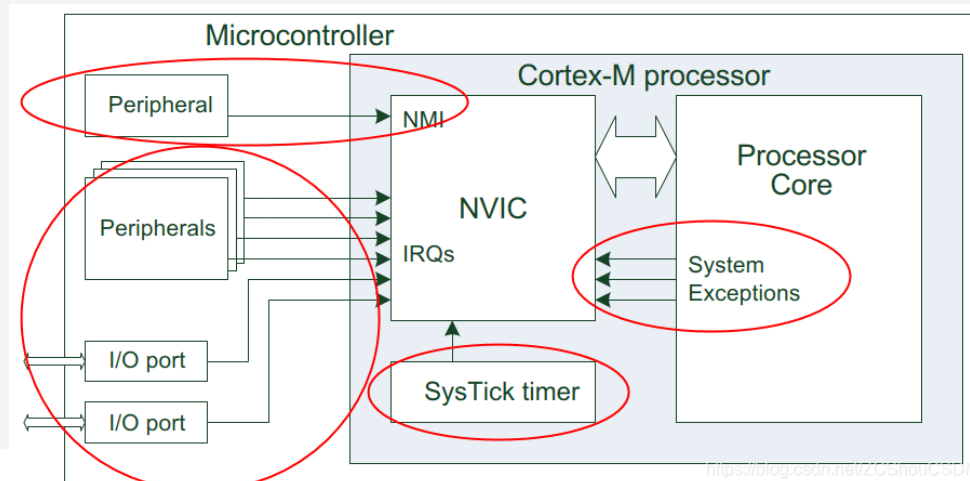
2.4.4 应用程序中的异常处理

ARM微处理器的异常处理

在一个正常的程序流程执行过程中，由内部或外部源产生的一个事件使正常的程序产生暂时的停止时，称之为异常。异常是由内部或外部源产生并引起处理器处理一个事件，例如一个外部的中断请求。在处理异常之前，当前处理器的状态必须保留，当异常处理完成之后，恢复保留的当前处理器状态，继续执行当前程序。多个异常同时发生时，处理器将会按固定的优先级进行处理。

ARM体系结构中的异常，与单片机的中断有相似之处，但异常与中断的概念并不完全等同，例如外部中断或试图执行未定义指令都会引起异常。

所有的 Cortex-M 内核都会包含一个用于中断处理的组件：NVIC（Nested Vectored Interrupt Controller，嵌套向量中断控制器）。它处理处理中断，还处理其他需要服务的事件（例如 SVC 指令），通常称为异常（按照 ARM 的说法，中断也是一种异常）。



2.4.1 ARM体系结构的异常类型

ARM体系结构支持7种类型的异常，异常类型、异常处理模式和优先级如下表所示。异常出现后，强制从异常类型对应的固定存储器地址开始执行程序。这些固定的地址称为异常向量。

异常类型	异常	进入模式	地址(异常向量)	优先级
复位	复位	管理模式	0x0000,0000	1(最高)
未定义指令	未定义指令	未定义模式	0x0000,0004	6(最低)
软件中断	软件中断	管理模式	0x0000,0008	6(最低)
指令预取中止	中止(预取指令)	中止模式	0x0000,000C	5
数据中止	中止(数据)	中止模式	0x0000,0010	2
IRQ(外部中断请求)	IRQ	IRQ	0x0000,0018	4
FIQ(快速中断请求)	FIQ	FIQ	0x0000,001C	3

2.4.2 异常类型的含义

(1) 复位

当处理器的复位电平有效时，产生复位异常，ARM处理器立刻停止执行当前指令。复位后，ARM处理器在禁止中断的管理模式下，程序跳转到复位异常处理程序处执行（从地址0x00000000或0xFFFF0000开始执行指令）。

(2) 未定义指令异常

当ARM处理器或协处理器遇到不能处理的指令时，产生未定义指令异常。当ARM处理器执行协处理器指令时，它必须等待任一外部协处理器应答后，才能真正执行这条指令。若协处理器没有响应，就会出现未定义指令异常。若试图执行未定义的指令，也会出现未定义指令异常。未定义指令异常可用于在没有物理协处理器（硬件）的系统上，对协处理器进行软件仿真，或在软件仿真时进行指令扩展。

异常类型的含义

(3) 软件中断异常 (SoftWare Interrupt, SWI)

软件中断异常由执行SWI指令产生，可使用该异常机制实现系统功能调用，用于用户模式下的程序调用特权操作指令，以请求特定的管理（操作系统）函数。

(4) 指令预取中止

若处理器预取指令的地址不存在，或该地址不允许当前指令访问，存储器会向处理器发出存储器中止 (Abort) 信号，但当预取的指令被执行时，才会产生指令预取中止异常。

(5) 数据中止 (数据访问存储器中止)

若处理器数据访问指令的地址不存在，或该地址不允许当前指令访问时，产生数据中止异常。存储器系统发出存储器中止信号。响应数据访问（加载或存储）激活中止，标记数据为无效。在后面的任何指令或异常改变CPU状态之前，数据中止异常发生。

异常类型的含义

(6) 外部中断请求 (IRQ) 异常

当处理器的外部中断请求引脚有效，且CPSR中的I位为0时，产生IRQ异常。系统的外设可通过该异常请求中断服务。IRQ异常的优先级比FIQ异常的低。当进入FIQ处理时，会屏蔽掉IRQ异常。

(7) 快速中断请求 (FIQ) 异常

当处理器的快速中断请求引脚有效，且CPSR中的F位为0时，产生FIQ异常。FIQ支持数据传送和通道处理，并有足够的私有寄存器。

Cortex-M3 和 Cortex-M4 的 NVIC 最多支持 240 个 IRQ(中断请求)、1 个不可屏蔽中断(NMI)、1 个 SysTick(滴答定时器)定时器中断和多个系统异常。

而 Cortex-M0 最多支持 32 个 IRQ、1 个不可屏蔽中断(NMI)、1 个 SysTick(滴答定时器)定时器中断和多个系统异常。

IRQ: 多数由定时器、IO 端口、通信接口等外设产生

NMI: 通常由看门狗定时器或者掉电检测器等外设产生

其他: 主要来自系统内核

2.4.3 异常的响应过程

当一个异常出现以后，ARM微处理器会执行以下几步操作：

- ① 将下一条指令的地址存入相应连接寄存器LR，以便程序在处理异常返回时能从正确的位置重新开始执行。若异常是从ARM状态进入，LR寄存器中保存的是下一条指令的地址（当前PC + 4或PC + 8，与异常的类型有关）；若异常是从Thumb状态进入，则在LR寄存器中保存当前PC的偏移量；
- ② 将CPSR状态传送到相应的SPSR中；
- ③ 根据异常类型，强制设置CPSR的运行模式位；
- ④ 强制PC从相关的异常向量地址下一条指令执行，转到相应的异常处理程序。还可设置中断禁止位，禁止中断发生。

异常的响应过程

如果异常发生时，处理器处于Thumb状态，则当异常向量地址加载入PC时，处理器自动切换到ARM状态。

异常处理完毕之后，ARM微处理器会执行以下几步操作从异常返回：

- ① 将连接寄存器LR的值减去相应的偏移量后送到PC中。
- ② 将SPSR内容送回CPSR中。
- ③ 若在进入异常处理时设置了中断禁止位，要在此清除。

可以认为应用程序总是从复位异常处理程序开始执行的，因此复位异常处理程序不需要返回。

2.4.4 应用程序中的异常处理

在应用程序的设计中，异常处理采用的方式是在异常向量表中的特定位置放置一条跳转指令，跳转到异常处理程序。当ARM处理器发生异常时，程序计数器PC会被强制设置为对应的异常向量，从而跳转到异常处理程序，当异常处理完成以后，返回到主程序继续执行。

2.5 ARM的存储器结构

ARM体系结构允许使用现有的存储器和I/O器件进行各种存储器系统设计。

1. 地址空间

ARM体系结构使用 2^{32} 个字节的单一、线性地址空间。将字节地址做为无符号数看待，范围为 $0 \sim 2^{32} - 1$ 。

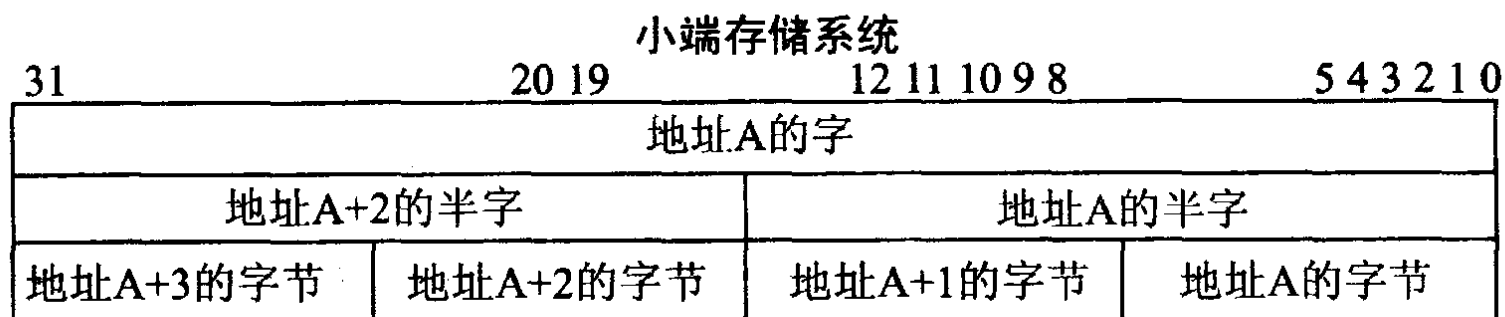
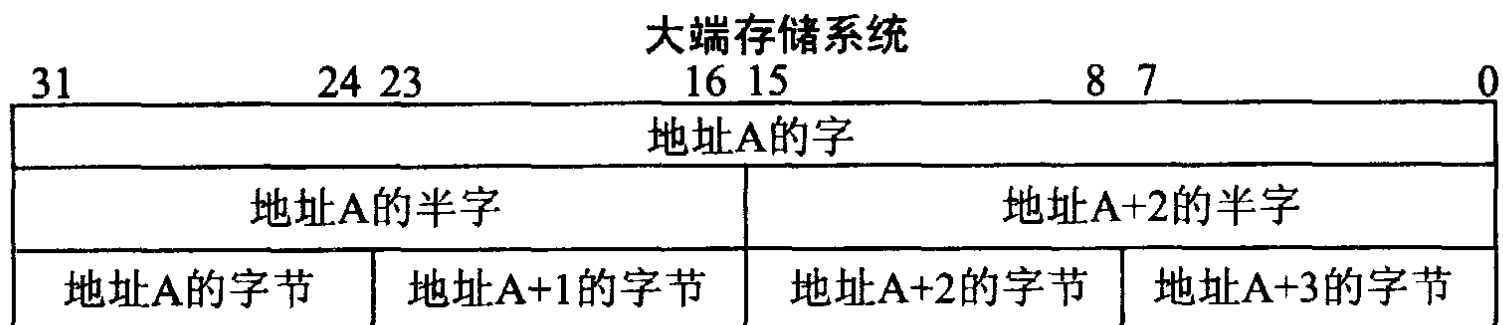
2. 存储器格式

对于字对齐的地址A，地址空间规则要求如下：

- 地址位于A的字由地址为A、A + 1、A + 2和A + 3的字节组成；
- 地址位于A的半字由地址为A和A + 1的字节组成；
- 地址位于A + 2的半字由地址为A + 2和A + 3的字节组成；
- 地址位于A的字由地址为A和A + 2的半字组成。

ARM存储系统可使用小端或大端存储两种方法，大端存储和小端存储格式如下图所示。

大端存储和小端存储格式



ARM体系结构通常希望所有的存储器访问能适当地对齐。特别是用于字访问的地址通常应当字对齐，用于半字访问的地址通常应当半字对齐。未按这种方式对齐的存储器访问称作**非对齐的存储器访问**。

ARM的存储器结构

3.ARM存储器结构

ARM处理器有的带有指令Cache和数据Cache，但不带有片内RAM和片内ROM。系统所需的RAM和ROM（包括Flash）都通过总线外接。由于系统的地址范围较大（ $2^{32} = 4\text{GB}$ ），有的片内还带有存储器管理单元MMU（Memory Management Unit）。ARM架构处理器还允许外接PCMCIA。

4.存储器映射I/O

ARM系统使用存储器映射I/O。I/O口使用特定的存储器地址，当从这些地址加载（用于输入）或向这些地址存储（用于输出）时，完成I/O功能。加载和存储也可用于执行控制功能，代替或者附加到正常的输入或输出功能。

2.7 ARM微处理器的接口

2.7.1 ARM协处理器接口

2.7.2 ARM AMBA接口

2.7.3 ARM I/O结构

2.7.4 ARM JTAG调试接口

2.7.1 ARM协处理器接口

为便于SoC的设计，ARM可通过协处理器(CP)支持通用指令集的扩充，增加系统的功能。逻辑上，ARM可扩展16个(CP15 ~ CP0)协处理器，其中：CP15为系统控制，CP14为调试控制器，CP7 ~ 4为用户控制器，CP13 ~ 8和CP3 ~ 0保留。

每个CP有16个寄存器。例如MMU和保护单元的系统控制都采用CP15；JTAG调试中的CP为CP14，即调试通信通道DCC。

ARM处理器内核与协处理器接口有以下4类。

- ① 时钟和时钟控制信号：MCLK、nWAIT、nRESET；
- ② 流水线跟随信号：nMREQ、SEQ、nTRANS、nOPC、TBIT；
- ③ 应答信号：nCPI、CPA、CPB；
- ④ 数据信号：D[31:0]、DIN[31:0]、DOUT[31:0]。

ARM协处理器接口

协处理器的应答信号中：

- **nCPI**为ARM处理器至CPn协处理器信号，该信号低电压有效代表“协处理器指令”，表示ARM处理器内核标识了1条协处理器指令，希望协处理器去执行它。
- **CPA**为协处理器至ARM处理器内核信号，表示协处理器不存在，目前协处理器无能力执行指令。
- **CPB**为协处理器至ARM处理器内核信号，表示协处理器忙，还不能够开始执行指令。

ARM协处理器接口

协处理器也采用流水线结构，为了保证与ARM处理器内核中的流水线同步，在每一个协处理器内需有1个流水线跟随器（Pipeline Follower），用来跟踪ARM处理器内核流水线中的指令。由于ARM的Thumb指令集无协处理器指令，协处理器还必须监视TBIT信号的状态，以确保不把Thumb指令误解为ARM指令。

协处理器也采用Load/Store结构，用指令来执行寄存器的内部操作，从存储器取数据至寄存器或把寄存器中的数保存至存储器中，以及实现与ARM处理器内核中寄存器之间的数据传送。而这些指令都由协处理器指令来实现。

2. 7. 2 ARM AMBA接口

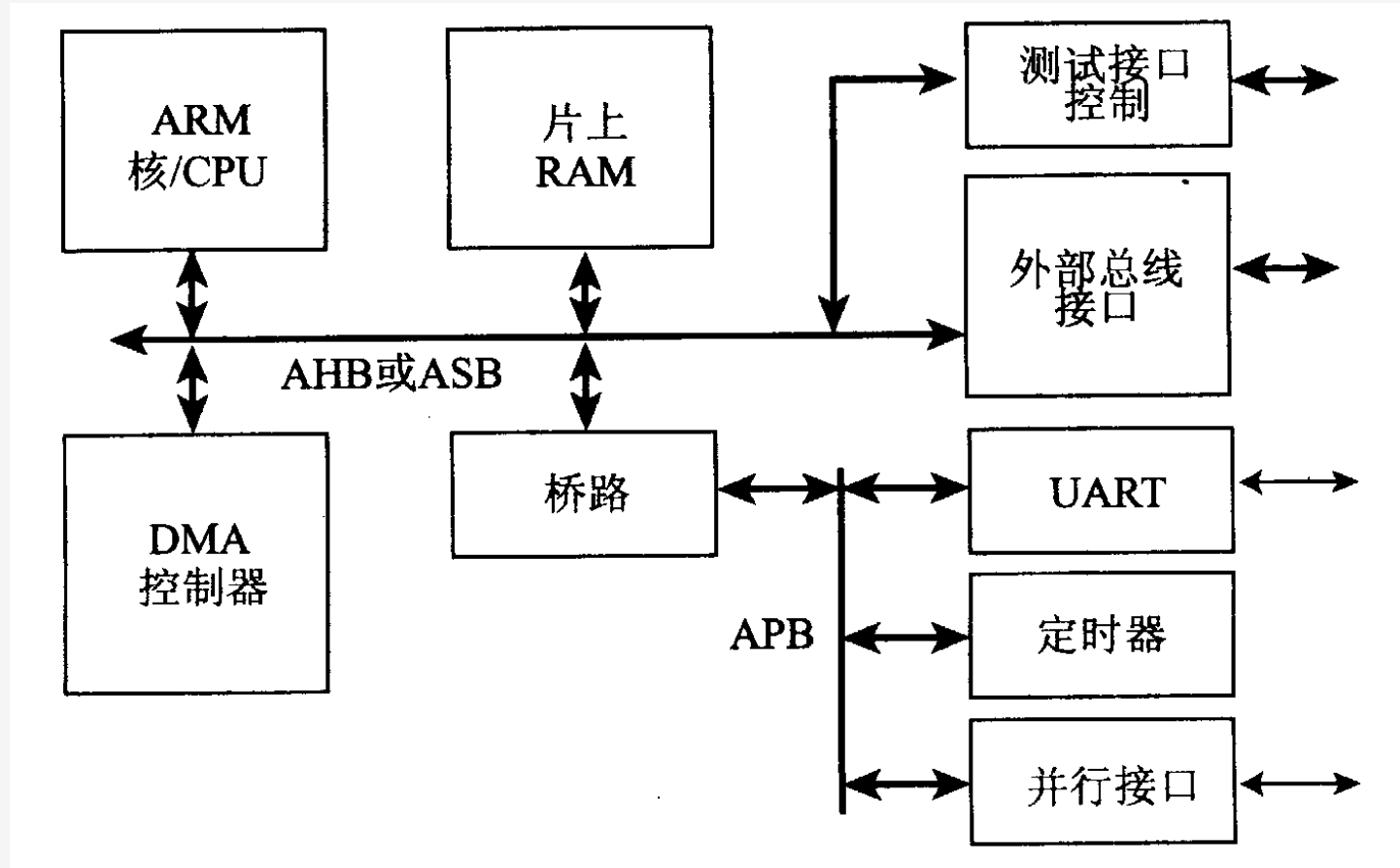
ARM处理器内核可通过先进的微控制器总线架构AMBA来扩展不同体系架构的宏单元及I/O部件。AMBA已成为事实上的片上总线OCB（On Chip Bus）标准。

AMBA有**AHB**（Advanced High-performance Bus，先进高性能总线）、**ASB**（Advanced System Bus，先进系统总线）和**APB**（Advanced Peripheral Bus，先进外围总线）等三类总线。

AHB不但支持突发方式的数据传送，还支持分离式总线事务处理，进一步提高总线的利用效率。在高性能的ARM架构系统中，AHB有逐步取代ASB的趋势，例如在ARM1020E处理器核中。

ASB是目前ARM常用的系统总线，用来连接高性能系统模块，支持突发方式数据传送。

一个基于AMBA的典型系统



ARM AMBA接口

APB为外围宏单元提供了简单的接口，也可以把APB看作ASB的余部。

AMBA通过测试接口控制器TIC（Test Interface Controller）提供了模块测试的途径，允许外部测试者作为ASB总线的主设备来分别测试AMBA上的各个模块。

AMBA中的宏单元也可以通过JTAG方式进行测试。虽然AMBA的测试方式通用性稍差些，但其通过并行口的测试比JTAG的测试代价也要低些。

2.7.3 ARM I/O结构

ARM处理器内核一般都没有I/O的部件和模块，ARM处理器中的I/O可通过AMBA总线来扩充。

ARM采用了存储器映像I/O的方式，即把I/O端口地址作为特殊的存储器地址。一般的I/O，如串行接口，它有若干个寄存器，包括发送数据寄存器（只写）、数据接收寄存器（只读）、控制寄存器、状态寄存器（只读）和中断允许寄存器等。这些寄存器都需相应的I/O端口地址。应注意的是存储器的单元可以重复读多次，其读出的值是一致的；而I/O设备的连续2次输入，其输入值可能不同。

在许多ARM体系结构中I/O单元对于用户是不可访问的，只可以通过系统管理调用或通过C的库函数来访问。

ARM I/O结构

ARM架构的处理器一般都没有DMA（直接存储器存取）部件，只有一些高档的ARM架构处理器才具有DMA的功能。

为了提高I/O的处理能力，对于一些要求I/O处理速率比较高的事件，系统安排了快速中断FIQ（Fast Interrupt reQuest），而对其余的I/O源仍安排一般中断IRQ。

为提高中断响应的速度，在设计中可以采用以下办法：

- 提供大量后备寄存器，在中断响应及返回时，作为保护现场和恢复现场的上下文切换（Context Switching）之用。
- 用片内RAM结构，可加速异常处理(包括中断)的进入时间。
- 快存Cache和地址变换后备缓冲器TLB（Translation Lookaside Buffer）采用锁住（Locked down）方式以确保临界代码段不受“不命中”的影响。

2.7.4 ARM JTAG调试接口

1. JTAG接口

JTAG (Joint Test Action Group, 联合测试行动小组) 是一种**国际标准测试协议**，主要用于**芯片内部测试**及对系统进行**仿真、调试**。JTAG技术是一种**嵌入式调试技术**，它在**芯片内部封装了专门的测试电路TAP(测试访问口)**，通过专用的**JTAG测试工具**对内部节点进行测试。目前大多数比较复杂的器件都支持JTAG协议，如ARM、DSP、FPGA器件等。

JTAG测试允许多个器件通过JTAG接口串联在一起，形成一个JTAG链，能实现对各个器件分别测试。JTAG接口还常用于实现**ISP(In-System Programmable在系统编程)**功能，如对Flash 器件进行编程等。

通过JTAG接口，可对芯片内部的所有部件进行访问，因而是开发调试嵌入式系统的一种简洁高效的手段。

标准的JTAG接口是4线式，**TMS(测试模式选择)**、**TCK(测试时钟)**、**TDI(测试数据输入)**和**TDO(测试数据输出)**。JTAG接口连接有**14针**和**20针**两种标准。

14针JTAG接口定义

引脚	名 称	描 述
1、13	VCC	接电源
2, 4, 6, 8, 10, 14	GND	接地
3	nTRST	测试系统复位信号
5	TDI	测试数据串行输入
7	TMS	测试模式选择
9	TCK	测试时钟
11	TDO	测试数据串行输出
12	NC	未连接

20针JTAG接口定义

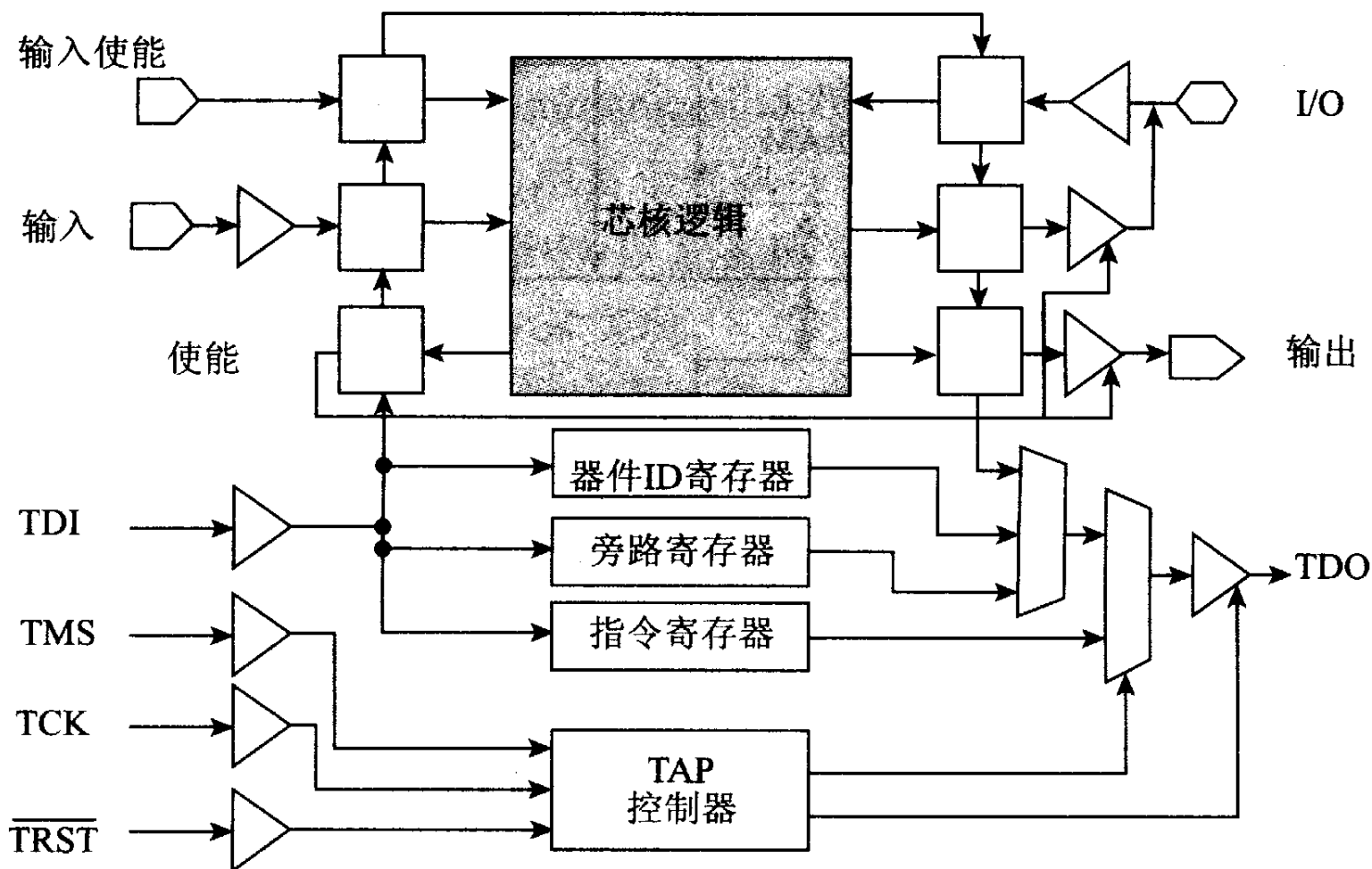
引脚	名 称	描 述
1	VTref	目标板参考电压, 接电源
2	VCC	接电源
3	nTRST	测试系统复位信号
4, 6, 8, 10, 12, 14, 16, 18, 20	GND	接地
5	TDI	测试数据串行输入
7	TMS	测试模式选择
9	TCK	测试时钟
11	RTCK	测试时钟返回信号
13	TDO	测试数据串行输出
15	nRESET	目标系统复位信号
17、19	NC	未连接

2. ARM JTAG调试接口

ARM JTAG调试接口的结构如下图所示。它由测试访问端口TAP (Test Access Port) 控制器、旁路 (Bypass) 寄存器、指令寄存器、数据寄存器以及与JTAG接口兼容的ARM架构处理器组成。

处理器的每个引脚都有一个移位寄存单元 (边界扫描单元 (BSC, Boundary Scan Cell)) , 它将JTAG电路与处理器核逻辑电路联系起来, 同时, 隔离了处理器核逻辑电路与芯片引脚。所有边界扫描单元构成了边界扫描寄存器BSR, 该寄存器电路仅在JTAG测试时有效, 在处理器核正常工作时无效。

JTAG调试接口示意图



(1) JTAG的控制寄存器

- ① **测试访问端口TAP控制器**对嵌入在ARM处理器核内部的测试功能电路进行访问控制，是一个同步状态机。通过测试模式选择TMS和时钟信号TCK来控制其状态转移，实现IEEE1149.1标准所确定的测试逻辑电路的工作时序；
- ② **指令寄存器**是串行移位寄存器，通过它可以串行输入执行各种操作的指令；
- ③ **数据寄存器组**是一组串行移位寄存器。操作指令被串行装入由当前指令所选择的数据寄存器，随着操作的进行，测试结果被串行移出。

JTAG测试信号、TAP状态机

(2) JTAG测试信号

JTAG包含有TRST、TCK、TMS、TDI、TDO五个测试信号；

JTAG可对同一电路板上多块芯片进行测试。TRST、TCK和TMS信号并行至各个芯片，而一块芯片的TDO接至下一芯片的TDI。

(3) TAP状态机

测试访问端口TAP控制器是一个16状态的有限状态机，为JTAG提供逻辑控制，控制进入JTAG结构中各种寄存器内数据的扫描与操作。在TCK同步时钟上升沿的TMS引脚的逻辑电压决定状态转移的过程。

由TDI引脚输入到器件的扫描信号有2个状态变化路径：用于指令移入至指令寄存器，或用于数据移入至相应的数据寄存器（该数据寄存器由当前指令确定）。

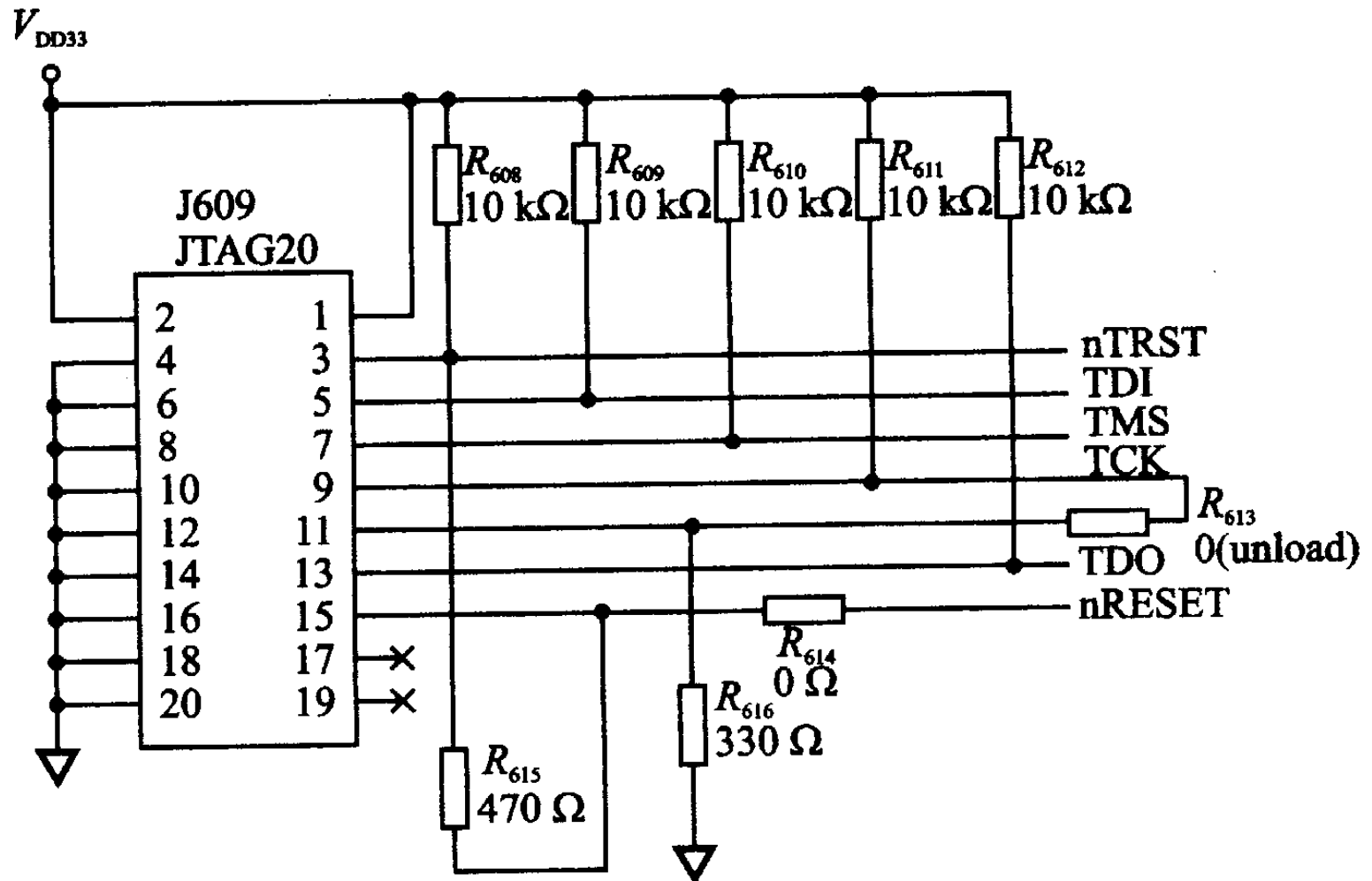
(4) JTAG接口控制指令

控制指令用于控制JTAG接口各种操作，控制指令包括**公用(Public)**指令和**私有(Private)**指令。最基本的**公用指令**有：

- **BYPASS**：旁路片上系统逻辑指令，用于未被测试的芯片，即把TDI与TPO旁路（1个时钟延迟）。
- **EXTEST**：片外电路测试指令，测试电路板上芯片间互连。
- **IDCODE**：读芯片ID码指令，用于识别电路板上的芯片。
- **INTEST**：片内测试指令，边界扫描寄存器位于TDI与TDO引脚之间，处理器核逻辑输入和输出状态被该寄存器捕获和控制。

ARM公司提供的标准20脚JTAG仿真调试接口电路如下图所示，可通过外部JTAG调试电缆或仿真器与开发系统连接调试。

JTAG仿真调试接口电路



嵌入式操作系统(Embedded OS)

Android

μC/OS

μC/Linux

Meego/Tizen

Symbian

iOS

Windows Phone/Mobile/Embedded

Windriver

Android

Android

❖以Linux为基础的半开源操作系统，主要用于移动终端，由Google领头成立的开放手持设备联盟 ([Open Handset Alliance - OHA](#))持续领导与开发中。

❖其内核属于Linux内核的分支，具有典型的Linux周期和功能，除此之外，Google为能让Linux在移动设备上良好的运行，对其进行修改和扩充。



Android硬件支持(1)

Android的开放性和可移植性，广泛用在电子产品上：

❖智能手机，上网本，平板电脑，PC，笔记本电脑，TV，机顶盒(STB)，MP3/MP4播放器，掌上游戏机，家用主机，电子手表，电子收音机，耳机，汽车设备，导航仪，CD/VCD/DVD播放机，以及其他设备。

Android大多搭载在使用了ARM架构的硬件设备上。

❖也有支持X86架构终端产品，Google TV；

❖Lenovo K800等智能手机和平板电脑；

❖也支持MIPS架构的智能手机和平板电脑。

Android硬件支持(2)

Apple公司的iOS设备, **iPhone, iPod Touch, iPad**均可安装Android, 且可以通过双系统启动工具**Open iBoot**或**iDroid**来运行Android。

微软公司的**Windows Mobile、Windows Phone**系列产品也同样可以。

另外Android亦已成功移植到**WebOS**系统的**HP TouchPad**以及**Meego**系统的**Nokia N9**等终端设备。

Android应用程序的开发

对于不同的软件开发包，使用的编程语言也不同。

❖在早期的Android应用程序开发中，通常通过在Android SDK中使用**Java**作为编程语言来开发应用程序。

❖开发者亦可以通过在Android NDK (Android Native开发包)中使用**C**或**C++语言**来作为编程语言开发应用程序。

❖同时Google还推出适合初学者编程使用的**Google Simple语言**，该语言类似微软公司的Visual Basic语言。

❖Google还推出Google App Inventor开发工具，该开发工具可以快速地构建应用程序，方便新手开发者。

Android版本

Version	Code name	Release date	API level	Distribution
2.2	Froyo	May 20, 2010	8	0.7%
2.3.3–2.3.7	Gingerbread	Feb. 9, 2011	10	13.6%
4.0.3–4.0.4	Ice Cream Sandwich	Dec.16, 2011	15	10.6%
4.1.x	Jelly Bean	Jul. 9, 2012	16	26.5%
4.2.x	Jelly Bean	Nov. 13, 2012	17	19.8%
4.3	Jelly Bean	Jul. 24, 2013	18	7.9%
4.4	KitKat	Oct. 31, 2013	19	20.9%

下一代版本： **Android L**； 专用版本： **Android Wear**

μC/OS

μC/OS是Micrium公司专门为计算机的嵌入式应用设计的，绝大部分代码是用C语言编写的。

❖ 一种公开源代码、结构小巧、具有可剥夺实时内核的实时操作系统，商业应用需要付费。

❖ 1992 年Jean J. Labrosse 在《嵌入式系统编程》杂志的5月和6月刊上刊登的文章连载发布，并把μC/OS 的源码发布在该杂志的BBS上。



μC/OS-III

μC/OS-III是可升级可固化抢占的基于优先级的实时内核。

❖对任务个数无限制。支持现代实时内核的大部分功能，例如：资源管理，同步，任务间的通信等等。

❖独有的特色功能，例如：完备的运行时间测量性能，直接发送信号或消息到任务，任务可同时等待多个内核对象等。

μC/OS-III 最主要的目标是提供一流的实时内核，以适应快速更新的嵌入式产品。

❖使用像μC/OS-III 具有雄厚基础和稳定框架的商业实时内核，能够处理日益复杂的嵌入式设计。